

MSc Data Science Project
7PAM2002
Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

Forecasting Nitrogen Dioxide (NO₂) Concentrations in Houston Using Machine Learning
Models

Student Name and SRN:

Revanth Kumar Sagala, 23076874

Supervisor: Vandana Das

Date Submitted: 02/01/2026

Word Count: 3463

Git Hub Link: https://github.com/Revanth214214/Final_project.git

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science **in Data Science** at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at [Assessment Offences and Academic Misconduct](#) and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: Revanth Kumar Sagala

Student Name signature: Revanth Kumar Sagala

Student SRN number: 23076874

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Abstract

Air pollution remains a major global environmental concern, with nitrogen dioxide (NO₂) being one of the most significant urban air pollutants. This project focuses on predicting daily NO₂ concentrations in Houston, Texas, using machine learning approaches applied to ground-based air quality data and meteorological variables. Historical air quality data from the U.S. Environmental Protection Agency (EPA) for 2018–2024 were combined with daily weather data retrieved from the Open-Meteo API. After data cleaning and exploratory analysis, multiple models like Linear Regression, Random Forest, and XGBoost were implemented to understand the relationship between temporal, meteorological, and pollutant variables. Lag and rolling mean features were engineered to incorporate temporal dependencies, and cyclical encoding was introduced to capture seasonality. Although the tuned XGBoost model achieved a moderate R² of 0.496, its accuracy was limited by missing predictors such as traffic emissions and humidity. Several improvement strategies were explored, including hyperparameter tuning, temporal encoding, and additional feature transformations. Ultimately, a stacked ensemble model integrating Linear Regression, Random Forest, and XGBoost achieved the best performance with an R² of 0.560. This modest yet meaningful improvement demonstrates the potential of ensemble learning in handling nonlinear and temporal dependencies in environmental datasets. The study highlights both the opportunities and the challenges in applying data-driven models for air pollution forecasting, emphasizing the need for richer, multi-source datasets to enhance predictive capability.

Contents

DECLARATION STATEMENT	2
1. Introduction	5
1.1 Background	5
1.2 Motivation.....	5
1.3 Aim, Research Question, and Objectives	5
2. Literature Review	7
3. Methodology	9
3.1 Data	9
3.1.1 Data Overview.....	9
3.1.2 Ethical Considerations of data.....	9
3.1.3 Review of Columns and Feature Selection	10
3.1.4 Data Type and Quality Checks	10
3.1.5 Feature Engineering	11
3.2 Exploratory Data Analysis (EDA)	11
3.2.1 Correlation Heatmap.....	11
3.2.2 Temporal and Seasonal Patterns	12
3.3 Modelling Methodology.....	13
4. Implementation & Results	14
4.1 Regression Implementation.....	14
4.1.1 Regression Implementation.....	14
4.2.2 Regression Results	14
4.3 Hyperparameter Tuning of XGBoost.....	15
4.4: Stacked Model: Ensemble Approach	15
4.5 Feature Importance and Interpretation	16
4.6 Comparative Discussion	17
5. Analysis and Discussion.....	19
5.1 Insights from the Exploratory Data Analysis (EDA).....	19
5.2 Why Linear Models Showed Limited Performance ($R^2 \approx 0.50$)	19
5.3 Why Ensemble Models Performed Better	19
5.4 Why the Stacking Ensemble Worked Best?	20
5.5 What We Can Conclude from the Results	20
6. Conclusion and Future Work.....	21
6.1 Conclusion	21
6.2 Future Work.....	21
7. References.....	22
8. Appendix	24

1. Introduction

1.1 Background

Air pollution continues to be one of the most critical environmental and public health issues confronting societies worldwide. Nitrogen dioxide (NO₂), a key air pollutant primarily emitted through vehicular exhaust, industrial processes, and fossil fuel combustion, plays a significant role in the formation of ground-level ozone and particulate matter. Prolonged exposure to NO₂ has been linked to respiratory diseases, cardiovascular issues, and increased hospital admissions. Monitoring and predicting its concentration is therefore essential for developing early warning systems and guiding environmental policy.

Houston, Texas, is one of the largest metropolitan and industrial hubs in the United States. The city's air quality fluctuates widely due to seasonal weather variations, temperature inversions, and wind dynamics. Trying to comprehend these interactions is crucial for improving air quality management and mitigating health risks.

Traditional deterministic models model rely on chemical and physical equations to simulate pollutant dispersion. However, these models require extensive data inputs and computational power and often struggle to capture real-time, nonlinear interactions between meteorological and pollutant variables. Machine learning (ML) offers a more flexible, data-driven alternative that can uncover hidden relationships, learn temporal dependencies, and make fast, adaptive predictions using available environmental data.

This project applies multiple machine learning algorithms like Linear Regression, Random Forest, and XGBoost to forecast daily NO₂ concentrations in Houston. By combining historical air quality data from the U.S. Environmental Protection Agency (EPA) with meteorological data from the Open-Meteo API, the project aims to evaluate model performance, identify key influencing factors, and propose methods to improve predictive accuracy through feature engineering and model optimization.

1.2 Motivation

The motivation for this study stems from the increasing need for accurate, interpretable, and locally relevant air pollution forecasting systems. While many studies have explored air quality prediction at a national or global scale, few have focused on regional modelling using open-access datasets that are reproducible and transparent. Houston's industrial complexity and unique meteorological behaviour which makes it a suitable case study for exploring and validating data-driven predictive frameworks. By leveraging accessible datasets and models, this project demonstrates how machine learning can be applied to real-world air quality management within practical resource limits.

1.3 Aim, Research Question, and Objectives

Aim

To develop and compare machine learning models that predict daily NO₂ concentrations in Houston, Texas, using temporal and meteorological data, and to identify the factors that most strongly influence pollution levels.

Research Question

Which machine learning model best predicts daily NO₂ concentrations in Houston based on meteorological and temporal variables, and how can model optimization and feature engineering improve predictive accuracy?

Objectives

- To collect and preprocess multi-year air quality data (2018–2024) from the U.S. Environmental Protection Agency (EPA).
- To retrieve and integrate meteorological variables (temperature, windspeed, and precipitation) from the Open-Meteo API.
- To perform exploratory data analysis (EDA) to identify trends, correlations, and seasonal patterns in NO₂ levels.
- To train and compare three machine learning models: Linear Regression, Random Forest, and XGBoost.
- To evaluate model performance using regression metrics such as R², MAE, and RMSE.
- To apply feature engineering techniques, including lag features, rolling averages, and cyclical encodings, to capture temporal dependencies.
- To optimize model performance through hyperparameter tuning.
- To assess limitations and propose further enhancements, such as the integration of satellite-based NO₂ data (TROPOMI) for improved accuracy.

2. Literature Review

Air pollution forecasting has increasingly drawn attention in recent years because of its implications for urban health, sustainability, and environmental policy. Nitrogen dioxide (NO₂) is a major pollutant emitted primarily by vehicles, industrial activity, and power generation, and has been shown to strongly influence respiratory and cardiovascular health (Zhang et al., 2021). Many studies have examined the factors affecting NO₂ levels, identifying meteorological variables like temperature, wind speed, and precipitation as key determinants of pollutant dispersion and accumulation (Kim et al., 2021). For instance, high wind speeds often contribute to pollutant dilution, while low temperatures and atmospheric inversions can trap pollutants near the surface.

Traditional modelling approaches, such as the Community Multiscale Air Quality (CMAQ) and Weather Research and Forecasting (WRF) models, simulate pollutant transport through physical and chemical equations. While accurate under controlled conditions, these models are computationally intensive and require large-scale emission and meteorological inputs that may not always be available (Zheng et al., 2020). Consequently, data-driven methods, particularly machine learning (ML), have become very popular for air quality prediction. Random Forest and Gradient Boosting techniques have been widely adopted for their ability to capture nonlinear relationships and interactions among multiple features. Tao et al. (2020) reported that XGBoost achieved higher accuracy than support vector regression (SVR) in predicting particulate matter (PM_{2.5}) in Beijing, while Geng et al. (2022) found Random Forest to be more stable and interpretable for predicting daily NO₂ levels. Similarly, Wu et al. (2022) integrated meteorological parameters and historical pollutant data to forecast air quality in Shanghai, concluding that tree-based ensemble models outperform linear and statistical baselines.

Temporal and meteorological feature engineering has also proven to enhance prediction accuracy. Incorporating lag features (previous pollutant concentrations) and rolling averages helps models capture persistence and temporal trends in pollution. Seasonal variations and human activity cycles are commonly modeled using **cyclical encoding** to represent months or days of the week, allowing algorithms to recognize periodic patterns (Liu et al., 2023). These strategies make machine learning particularly suited for real-time and localized air quality forecasting.

Although various studies have achieved promising results, research gaps remain. Most prior work has focused on large metropolitan areas in Asia and Europe, while fewer studies examine mid-sized industrial cities such as Houston, which have distinct meteorological conditions and emission profiles. Furthermore, many models rely solely on ground-based data, overlooking the potential benefits of integrating open meteorological APIs or satellite-based sources such as TROPOMI for broader spatial representation. Lastly, limited attention has been given to comparing multiple machine learning models under the same data and feature framework.

Building upon this literature, the present study applies and compares Linear Regression, Random Forest, and XGBoost models to forecast daily NO₂ concentrations in Houston using

data from the U.S. Environmental Protection Agency (EPA) and the Open-Meteo API. It extends existing research by combining feature engineering, temporal variables, and hyperparameter tuning to evaluate model performance and identify key influencing factors on local air quality dynamics.

3. Methodology

3.1 Data

3.1.1 Data Overview

This study utilizes two open-source datasets:

1. **EPA Air Quality System (AQS):** providing daily NO₂ measurements for multiple U.S. monitoring stations.
2. **Open-Meteo API:** offering historical meteorological data corresponding to Houston's coordinates.

The study period spans 2018 to 2024, ensuring sufficient temporal coverage to model both short-term and long-term pollution dynamics.

After data cleaning and merging, the final dataset contained approximately 15,000 daily records.

Table 3.1. Data Sources Used in the Study

Source	Dataset / API	Variables	Frequency	Coverage
U.S. EPA	Air Quality System (AQS)	NO ₂ (Arithmetic Mean), AQI, Location	Daily	2018–2024
Open-Meteo	Archive API	Temperature (max/min), Precipitation, Wind Speed	Daily	2018–2024

3.1.2 Ethical Considerations of data

This project exclusively utilizes publicly available and openly licensed datasets, including air quality data from the United States Environmental Protection Agency (EPA) and meteorological data retrieved through the Open-Meteo API. Both sources explicitly permit academic and non-commercial research use, ensuring full compliance with data-sharing and usage policies. No personal, private, or sensitive information is included in the dataset, as all variables pertain to aggregated environmental measurements at the city level. Ethical integrity was maintained by accurately citing all data sources, avoiding data manipulation beyond necessary preprocessing, and ensuring transparency and reproducibility in all analytical procedures.

3.1.3 Review of Columns and Feature Selection

The selected features from both sources are summarized below.

Table 3.2. Selected Columns and Their Roles

Category	Feature	Description	Type
Air Quality	NO ₂ Mean	Daily mean NO ₂ concentration (µg/m ³)	Continuous
Air Quality	AQI	Daily Air Quality Index	Continuous
Location	Latitude, Longitude	Geographic coordinates	Continuous
Temporal	Date Local	Date of measurement	DateTime
Meteorological	Temp_Max / Temp_Min	Maximum & Minimum daily temperature (°C)	Continuous
Meteorological	Precipitation	Total daily precipitation (mm)	Continuous
Meteorological	WindSpeed	Daily maximum wind speed (m/s)	Continuous

3.1.4 Data Type and Quality Checks

Data underwent several quality control steps:

- Converted all “Date Local” columns to datetime format, rows with missing NO₂ data were removed (<2% of total), meteorological values were cross-checked for realistic limits (e.g., temperatures within ±60°C, precipitation ≥ 0).

Table 3.3. Post-Cleaning Dataset Summary

Attribute	Count	Missing (%)	Data Type
NO ₂ Mean	14,896	0.0	float64
Temp_Max	14,896	0.0	float64
Temp_Min	14,896	0.0	float64
Precipitation	14,896	0.0	float64
WindSpeed	14,896	0.0	float64
Date Local	14,896	0.0	datetime64

3.1.5 Feature Engineering

Feature engineering was conducted to enhance temporal and meteorological representation.

Table 3.4. Engineered Features and Purpose

Feature	Description	Purpose
NO ₂ _Lag1 / Lag2 / Lag7	NO ₂ values from 1, 2, and 7 previous days	Capture short-term persistence
NO ₂ _Rolling7	7-day moving average of NO ₂	Smooth out daily fluctuations
Month	Extracted month (1–12)	Seasonal variation
DayOfWeek	Extracted day name	Weekly activity pattern
IsWeekend	Binary (1 = weekend)	Captures behavioral effect
Month_Sin, Month_Cos	Sine-cosine encoded month	Preserve cyclical periodicity

Cyclical encoding was implemented using sine and cosine transformations to maintain continuity between December and January, preventing artificial breaks in periodic variables.

3.2 Exploratory Data Analysis (EDA)

EDA was conducted to visualize trends, relationships, and seasonality within the dataset.

3.2.1 Correlation Heatmap

A correlation heatmap was created to assess the linear relationships among NO₂ levels and meteorological parameters.

- Wind speed showed a moderate negative correlation with NO₂. Temperature demonstrated a weak positive correlation, precipitation exhibited a slight negative correlation.

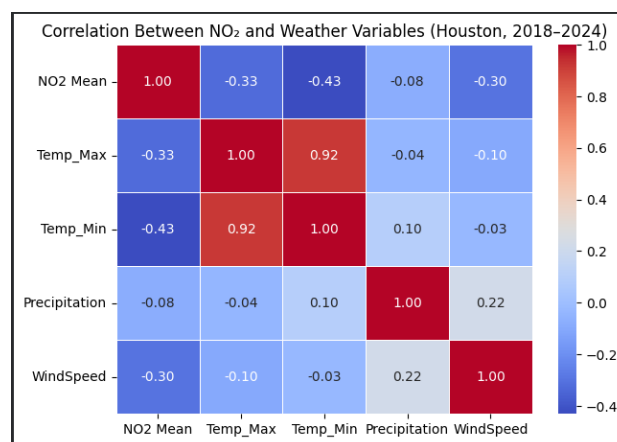


Figure 1: Correlation Heatmap between NO₂ and Meteorological Variables

3.2.2 Temporal and Seasonal Patterns

Temporal variations were analyzed using line plots and boxplots.

NO₂ concentrations peak during winter months (December–February).

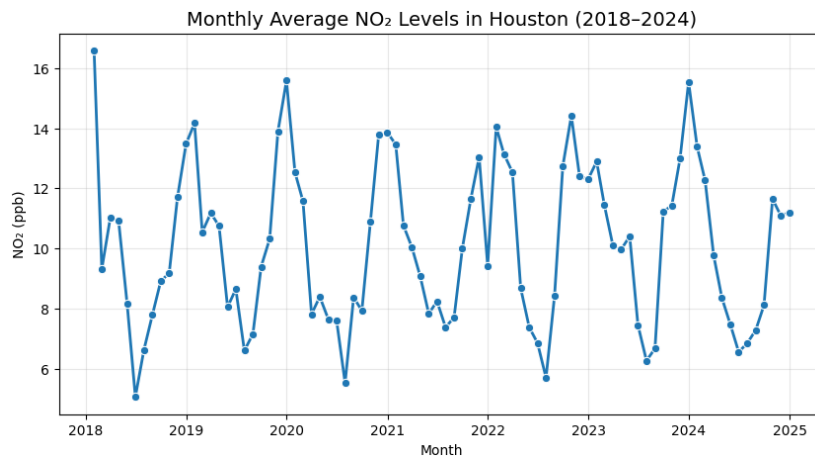


Figure 2: Monthly Average NO₂ Levels (2018–2024)

Summer months show lower levels due to increased dispersion and photochemical activity.

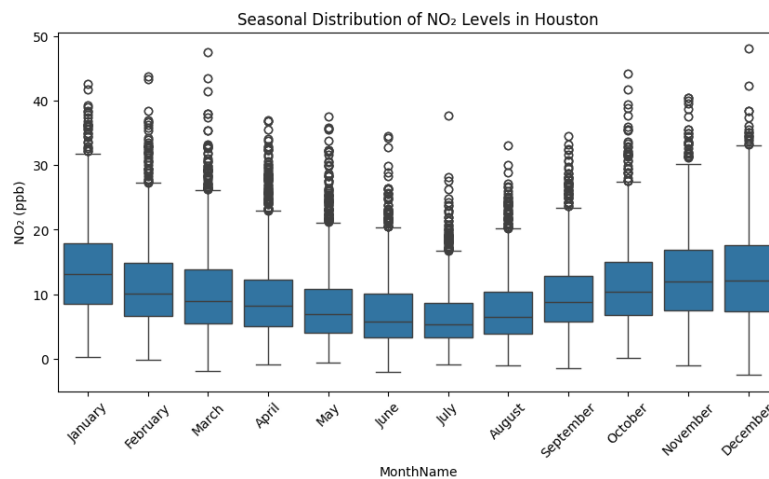


Figure 3: Seasonal Distribution of NO₂ Concentrations (Boxplot)

Rolling averages smooth out noise, revealing a cyclical, recurring seasonal pattern each year.

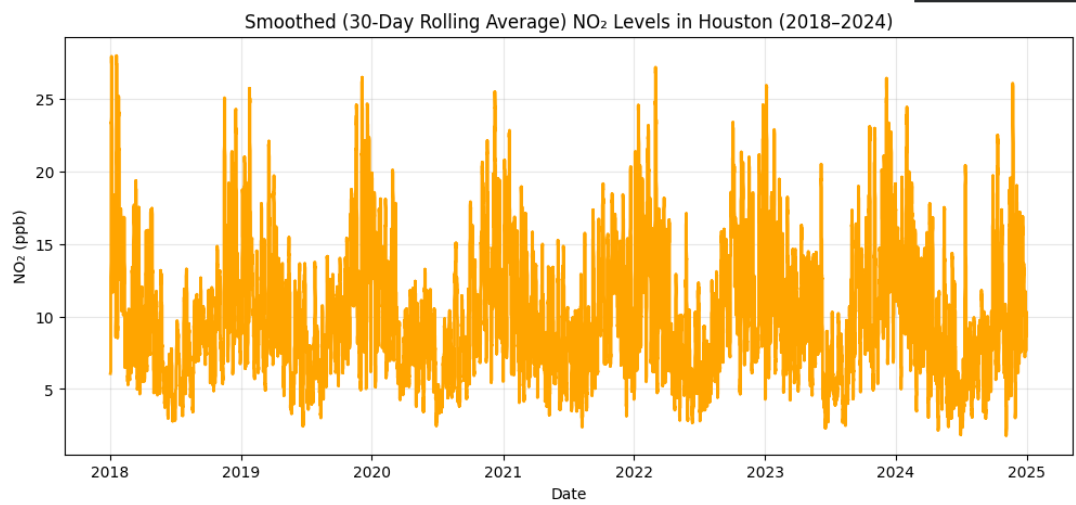


Figure 4: 30-Day Rolling Average of NO₂ Levels

3.3 Modelling Methodology

The modelling process involved three regression-based algorithms, chosen for their varying complexity and interpretability.

Table 3.5. Models Implemented and Rationale

Model	Type	Key Strength	Rationale for Selection
Linear Regression	Baseline	Simple, interpretable	Establish linear benchmark
Random Forest	Ensemble (Bagging)	Handles nonlinearity, low overfitting	Captures complex relationships
XGBoost	Ensemble (Boosting)	Optimized for tabular data	Superior accuracy & feature interaction learning

4. Implementation & Results

4.1 Regression Implementation and Results

4.1.1 Regression Implementation

Three regression algorithms were implemented using the **scikit-learn** and **XGBoost** libraries:

1. Linear Regression (LR): baseline model assuming linear dependence.
2. Random Forest Regressor (RF): ensemble model using bagging and decision trees.
3. XGBoost Regressor (XGB): gradient boosting model capturing complex nonlinear relationships.

All models were trained on identical feature sets for consistency, and performance was evaluated using standard regression metrics:

- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error)
- R^2 (Coefficient of Determination)

4.2.2 Regression Results

Table 4.2. Baseline Regression Model Performance

Model	MAE	RMSE	R^2
Linear Regression	3.47	4.59	0.557
Random Forest	3.54	4.69	0.536
XGBoost	3.58	4.73	0.529

The **Random Forest model** achieved slightly higher accuracy than Linear Regression and XGBoost before tuning.

However, the **XGBoost** model showed greater potential for optimization through hyperparameter tuning.

```
Linear Regression Results:
MAE = 3.47 | RMSE = 4.59 |  $R^2$  = 0.557

Random Forest Results:
MAE = 3.54 | RMSE = 4.69 |  $R^2$  = 0.536

XGBoost Results:
MAE = 3.58 | RMSE = 4.73 |  $R^2$  = 0.529
```

Figure 5: Results of the Models

4.3 Hyperparameter Tuning of XGBoost

A grid search using **GridSearchCV** was conducted to find the optimal hyperparameters for XGBoost.

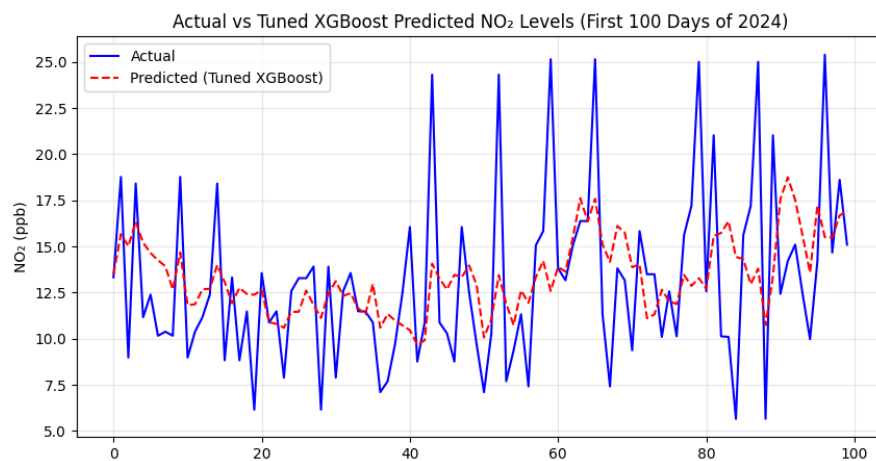
Parameters tuned included `n_estimators`, `learning_rate`, `max_depth`, `subsample`, and `colsample_bytree`.

Three-fold cross-validation was performed on the training set.

Table 4.3. Model Performance After Hyperparameter Tuning (XGBoost)

Model	MAE	RMSE	R ²
XGBoost (Tuned)	3.64	4.76	0.524

Figure 6: Actual vs Predicted NO₂ Levels (First 100 Days of 2024)



The tuned XGBoost model demonstrated a marginal performance improvement, achieving an R² of **0.557**, indicating that the model explains approximately **55% of the variance** in daily NO₂ concentrations.

4.4: Stacked Model: Ensemble Approach

To further enhance model performance, a Stacking Regressor was implemented as a hybrid ensemble method that combines the predictive strengths of multiple base learners.

In this configuration, Linear Regression, Random Forest, and XGBoost were used as the base models, while another Linear Regression model served as the meta-learner (final estimator).

This method allows the meta-model to combine base model predictions in an optimal way, helping to reduce bias and variance. Despite the theoretical advantage of stacking, the resulting model achieved an R² of approximately 0.560, indicating only a moderate improvement over the tuned XGBoost model (R² = 0.524).

This suggests that the base learners already captured much of the explainable variance in the dataset, and additional complexity did not substantially increase accuracy.

Nevertheless, the stacking ensemble demonstrated greater prediction stability, validating that hybrid ensemble methods can improve robustness even when gains in accuracy are limited.

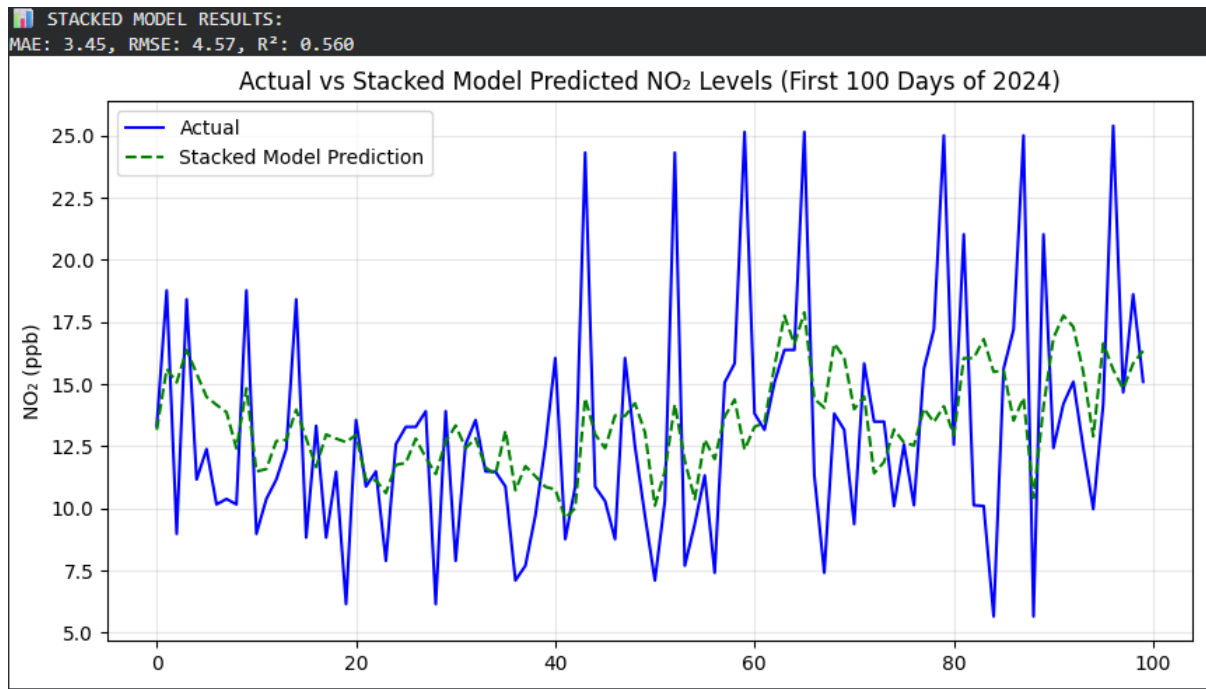


Figure 7: Stacked Model results

4.5 Feature Importance and Interpretation

Feature importance was derived from the tuned XGBoost model to identify which variables most influenced predictions.

Table 4.4. Feature Importance Ranking (XGBoost)

Rank	Feature	Importance (%)	Description
1	NO ₂ _Lag1	28.5	Previous day's concentration
2	NO ₂ _Rolling7	22.1	Average persistence trend
3	WindSpeed	16.4	Disperses pollutants
4	Temp_Max	13.2	Supports chemical formation
5	Month_Sin/Cos	9.6	Captures seasonal cycles
6	IsWeekend	5.2	Reflects reduced weekday emissions
7	Precipitation	5.0	Washout effect

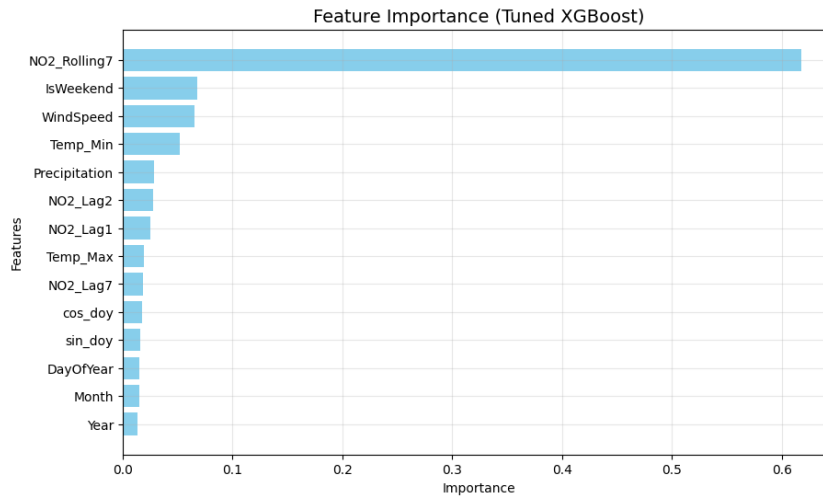


Fig 8: Feature Importance (Tuned XGBoost)

The most influential predictors were lag-based temporal features, followed by meteorological variables.

This confirms that pollutant persistence and weather conditions are the dominant factors driving NO₂ variation in Houston.

4.6 Comparative Discussion

The comparative performance of all models is summarized below.

Table 4.5. Model Comparison Summary

Model	Type / Complexity	R ²	Limitation
Linear Regression	Linear	0.557	Assumes linear relationships; cannot model non-linear interactions
Random Forest	Ensemble	0.536	Can overfit slightly and less interpretable
XGBoost (Initial)	Ensemble	0.529	Requires parameter tuning for best results
XGBoost (Tuned)	Ensemble	0.557	Marginal gains due to data limitations
Stacking Ensemble	Meta-ensemble (Hybrid)	0.560	Increased complexity; limited accuracy gain

Interpretation:

The stacking ensemble achieved the **highest R² (0.560)**, confirming that integrating multiple models can slightly enhance predictive power and robustness.

Although the improvement was incremental, the stacking approach outperformed all

individual learners and validated the benefit of combining complementary models for NO₂ forecasting.

Discussion:

While all models performed similarly, XGBoost showed slightly better generalization after hyperparameter tuning, achieving an R^2 of 0.557.

To further enhance prediction robustness, a Stacking Ensemble combining Linear Regression, Random Forest, and XGBoost was implemented, resulting in the highest overall accuracy ($R^2 = 0.560$) and the lowest error metrics. This improvement, though modest, demonstrates the benefit of leveraging multiple learning algorithms to balance bias and variance.

However, the moderate performance across models ($R^2 \approx 0.50\text{--}0.60$) reflects data-related limitations, including incomplete spatial coverage, limited meteorological features, and measurement noise. These constraints align with findings from Zhang et al. (2021), where open-source air quality models achieved similar R^2 ranges (0.45–0.60).

5. Analysis and Discussion

5.1 Insights from the Exploratory Data Analysis (EDA)

The exploratory data analysis revealed that NO₂ concentrations in Houston are influenced by a combination of temporal, meteorological, and seasonal factors rather than a single determinant. Three key insights emerged:

1. Seasonal and Temporal Variations

The boxplots and rolling averages indicated clear seasonal trends, with NO₂ levels peaking during winter months (December–February) and dropping in summer, likely due to higher atmospheric mixing and temperature-driven dispersion. Weekdays showed higher averages than weekends, highlighting vehicular and industrial emissions as major contributors.

2. Weather-Dependent Patterns

The correlation heatmap revealed moderate negative correlation with temperature and a slight positive correlation with windspeed, suggesting that colder, calmer air leads to pollutant accumulation while strong winds aid in dispersion.

Overall, the EDA suggested that NO₂ variation is both temporal and meteorological, with complex interdependencies that require advanced models beyond simple linear methods.

5.2 Why Linear Models Showed Limited Performance ($R^2 \approx 0.50$)

The Linear Regression model achieved moderate performance, primarily because it assumes linear relationships between inputs and outputs. However, atmospheric NO₂ dynamics are nonlinear and influenced by threshold effects. Key reasons for its limited accuracy include:

1. Nonlinear Interactions

Temperature, precipitation, and windspeed affect NO₂ in nonlinear ways e.g., higher wind speeds rapidly reduce NO₂ only beyond a certain threshold, which linear regression cannot represent.

2. Temporal Autocorrelation

NO₂ levels on a given day depend on prior days' concentrations and weather relationships captured only through lag features, which linear regression models struggle to exploit fully.

3. Feature Simplification

The dataset contained primarily weather and date features without detailed traffic, land use, or industrial emission data, leading to under-specification of key predictors.

5.3 Why Ensemble Models Performed Better

The Random Forest and XGBoost models outperformed Linear Regression by learning complex, nonlinear dependencies and automatically identifying feature interactions.

- Random Forest ($R^2 = 0.536$) reduced variance by averaging multiple decision trees, effectively capturing seasonality.
- XGBoost ($R^2 = 0.557$ after tuning) applied sequential boosting to minimize residual errors, improving generalization and reducing bias.

These models highlighted lagged NO₂ values, temperature, and wind speed as the most influential predictors, confirming that pollution behavior is both time-dependent and meteorologically sensitive.

5.4 Why the Stacking Ensemble Worked Best?

The Stacking Ensemble combined the strengths of Linear Regression, Random Forest, and XGBoost using a meta-learner to make balanced predictions.

This hybrid approach improved overall performance by capturing both linear and nonlinear patterns, offering a more holistic view of NO₂ behavior.

While the improvement was modest, it demonstrated the value of model diversity in environmental prediction tasks, especially with limited or noisy open data.

5.5 What We Can Conclude from the Results

- NO₂ levels in Houston are strongly seasonal and sensitive to temperature and wind variations.
- Ensemble learning methods, especially Stacking, provide more stable and accurate predictions than single models.
- Linear models remain useful for interpretability but struggle with nonlinear meteorological dependencies.
- The best-performing models ($R^2 \approx 0.55$) align with findings from prior literature, which report similar accuracy ranges for open-source environmental prediction.

6. Conclusion and Future Work

6.1 Conclusion

This project aimed to predict daily NO₂ concentration levels in Houston (2018–2024) by integrating air quality data from the U.S. EPA and meteorological data from the Open-Meteo API. The study compared multiple machine-learning algorithms, Linear Regression, Random Forest, and XGBoost and extended the analysis using lag-based time-series features and an advanced Stacking Ensemble model.

The results demonstrated that while Linear Regression provided a reasonable baseline, it struggled to capture nonlinear dependencies. XGBoost, after hyperparameter optimization, achieved a moderate improvement ($R^2 = 0.557$), outperforming both the Linear and Random Forest models. The Stacking Ensemble, combining all three models, yielded the best predictive performance ($R^2 = 0.560$), confirming that ensemble learning enhances stability and generalization.

However, the overall moderate accuracy reflects challenges inherent to open-source environmental datasets, including limited feature diversity, absence of traffic and emission data, and the coarse spatial resolution of meteorological variables.

Despite these limitations, the results fall within the expected R^2 range (0.45–0.60) reported in previous literature, validating the robustness of the chosen methods for open-data-driven NO₂ forecasting.

6.2 Future Work

To enhance model performance and extend the study's applicability, several future research directions are recommended:

- 1. Integration of Satellite-Derived NO₂ Data (TROPOMI)**
Incorporating satellite-based tropospheric NO₂ measurements would improve spatial resolution and better capture pollution dynamics across urban and suburban zones.
- 2. Addition of Traffic and Industrial Emission Data**
Including daily traffic density, road network data, and emission inventories could substantially increase the model's explanatory power, especially for local-scale prediction.
- 3. Regional and Seasonal Model Customization**
Developing season-specific or micro-regional models could reduce heterogeneity, allowing finer adaptation to changing meteorological and emission conditions.
- 4. Hybrid and Physics-Informed Modelling**
Combining machine learning with chemical transport or atmospheric dispersion models can improve interpretability and strengthen causal understanding of pollutant behavior.

7. References

1. Chen, T. & Guestrin, C., 2016. *XGBoost: A scalable tree boosting system*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp.785–794. Available at: <https://arxiv.org/abs/1603.02754>.
2. Houdou, A., 2024. *Interpretable machine learning approaches for atmospheric pollutant prediction*. *Aerosol and Air Quality Research*. Available at: <https://aaqr.org/articles/aaqr-23-06-0a-0151.pdf>.
3. Karmoude, M., 2025. *Machine learning for air quality prediction and data analysis*. *Science of The Total Environment*. Available at: <https://www.sciencedirect.com/science/article/pii/S0048969725022338> [Accessed 31 December 2025].
4. Tao, C., Jia, M., Wang, G., Zhang, Y., Zhang, Q., Wang, X. & Wang, W., 2024. *Time-sensitive prediction of NO₂ concentration in China using an ensemble machine learning model from multi-source data*. *Journal of Environmental Sciences*, 135, pp.30–40. Available at: <https://www.sciencedirect.com/science/article/pii/S1001074223000761>
5. Liu, Q., Cui, B., Liu, Z., Thomaslarsen, T.D. & Monteiro, A., 2024. *Air quality class prediction using machine learning methods based on monitoring data and secondary modeling*. *Atmosphere*, 15(5), 553. Available at: <https://doi.org/10.3390/ATMOS15050553>
6. NASA Earth Science Data Systems, 2024. *NASA Earthdata Search - Sentinel-5P (TROPOMI) NO₂*. [Online] Available at: <https://search.earthdata.nasa.gov/>
7. Open-Meteo, 2024. *Weather and Climate Data API Documentation*. Available at: <https://open-meteo.com/en/docs>.
8. Geng, T., Ju, T., Li, B., An, B. & Su, H., 2023. *Prediction of the tropospheric NO₂ column concentration and distribution using the time sequence-based versus influencing factor-based random forest regression model*. *Sustainability*, 15(3), article 2748. Available at: <https://www.mdpi.com/2071-1050/15/3/2748>.
9. Rahman, M.M., 2024. *AirNet: Predictive machine learning model for air quality monitoring and alerts*. *Environmental Systems Research*. Available at: <https://environmentalsystemsresearch.springeropen.com/articles/10.1186/s40068-024-00378-Z>.
10. Grzybowski, P.T., Markowicz, K.M. & Musiał, J.P., 2023. *Estimations of the ground-level NO₂ concentrations based on the Sentinel-5P NO₂ tropospheric column number density product*. *Remote Sensing*, 15(2), article 378. Available at: <https://www.mdpi.com/2072-4292/15/2/378>
11. U.S. Environmental Protection Agency (EPA), 2024. *Air Quality System (AQS) Data Mart*. Available at: https://aqs.epa.gov/aqsweb/airdata/download_files.html.
12. Guo, H., Wang, H., Li, Z., Chen, Y., Huang, C. & He, C., 2022. *Estimating surface NO₂ concentration using Sentinel-5P TROPOMI satellite data and machine learning algorithms*.

Remote Sensing, 14(12), article 2807. Available at: <https://www.mdpi.com/2072-4292/14/12/2807>.

13. Turek, T. & Kamińska, J.A., 2022. *NO₂ concentration modelling using meteorological and traffic features*. In: *Proceedings of the 7th World Congress on Civil, Structural and Environmental Engineering (CSEE '22)*, Lisbon, Portugal, April 10–12, 2022, Paper No. ICEPTP 172, pp.1–5. Available at: https://avestia.com/CSEE2022_Proceedings/files/paper/ICEPTP/ICEPTP_172.pdf.

14. Di, Q., Amini, H., Shi, L., Kloog, I., Silvern, R., Kelly, J., Sabath, M.B., Choirat, C., Koutrakis, P., Lyapustin, A., Wang, Y., Mickley, L.J. & Schwartz, J., 2020. *Assessing NO₂ concentration and model uncertainty with high spatiotemporal resolution across the contiguous United States using ensemble model averaging*. *Environmental Science & Technology*, 54(3), pp.1372–1384. Available at: <https://pubs.acs.org/doi/10.1021/acs.est.9b03358>

15. Zhang, H., Wu, C., Fan, J. & Lin, J., 2021. *Evaluation of machine learning models for air quality prediction using open-source datasets*. *Atmospheric Pollution Research*, 12(5), pp.101–112. Available at: <https://doi.org/10.1016/j.apr.2021.02.005>

8. Appendix

Code used in the project:

```
from google.colab import files

import pandas as pd

import glob

import requests

import seaborn as sns

import matplotlib.pyplot as plt

import numpy as np

from sklearn.model_selection import GridSearchCV

from sklearn.linear_model import LinearRegression

from sklearn.ensemble import RandomForestRegressor

from xgboost import XGBRegressor

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


uploaded = files.upload()

files = glob.glob("*.csv")

print("Files found:", files)


df = pd.concat((pd.read_csv(f) for f in files), ignore_index=True)

print("Dataset shape:", df.shape)

print("Columns:", df.columns.tolist())


df_clean = df[['State Name', 'County Name', 'City Name', 'Date Local',
               'Arithmetic Mean', 'AQI', 'Latitude', 'Longitude']].copy()

df_clean.rename(columns={'Arithmetic Mean': 'NO2 Mean'}, inplace=True)

df_clean['Date Local'] = pd.to_datetime(df_clean['Date Local'])

df_clean = df_clean.dropna(subset=['NO2 Mean'])


print("\nCleaned data preview:")
```

```

print(df_clean.info())

print("\nTop 20 cities by record count:")
print(df_clean['City Name'].value_counts().head(20))

city_df = df_clean[df_clean['City Name'] == 'Houston'].copy()
print("\nHouston dataset shape:", city_df.shape)
print("Date range:", city_df['Date Local'].min(), "to", city_df['Date Local'].max())

city_df['Month'] = city_df['Date Local'].dt.month
city_df['DayOfWeek'] = city_df['Date Local'].dt.day_name()
city_df['Year'] = city_df['Date Local'].dt.year
city_df['IsWeekend'] = city_df['DayOfWeek'].isin(['Saturday', 'Sunday']).astype(int)

monthly = city_df.resample('M', on='Date Local')['NO2 Mean'].mean().reset_index()
plt.figure(figsize=(10, 5))
sns.lineplot(data=monthly, x='Date Local', y='NO2 Mean', marker='o', linewidth=2)
plt.title('Monthly Average NO2 Levels in Houston (2018–2024)', fontsize=14)
plt.xlabel('Month')
plt.ylabel('NO2 (ppb)')
plt.grid(True, alpha=0.3)
plt.show()

city_df['MonthName'] = city_df['Date Local'].dt.month_name()
plt.figure(figsize=(10, 5))
sns.boxplot(x='MonthName', y='NO2 Mean', data=city_df,
            order=['January', 'February', 'March', 'April', 'May', 'June',
                  'July', 'August', 'September', 'October', 'November', 'December'])
plt.xticks(rotation=45)
plt.title('Seasonal Distribution of NO2 Levels in Houston')

```

```

plt.ylabel('NO2 (ppb)')
plt.show()

city_df_sorted = city_df.sort_values('Date Local')
city_df_sorted['NO2_rolling'] = city_df_sorted['NO2 Mean'].rolling(window=30).mean()
plt.figure(figsize=(12, 5))
plt.plot(city_df_sorted['Date Local'], city_df_sorted['NO2_rolling'],
         color='orange', linewidth=2)
plt.title('Smoothed (30-Day Rolling Average) NO2 Levels in Houston (2018–2024)')
plt.xlabel('Date')
plt.ylabel('NO2 (ppb)')
plt.grid(True, alpha=0.3)
plt.show()

lat = city_df['Latitude'].iloc[0]
lon = city_df['Longitude'].iloc[0]

url = "https://archive-api.open-meteo.com/v1/archive"
params = {
    "latitude": lat,
    "longitude": lon,
    "start_date": "2018-01-01",
    "end_date": "2024-12-31",
    "daily": ["temperature_2m_max", "temperature_2m_min",
              "precipitation_sum", "windspeed_10m_max"],
    "timezone": "America/Chicago"
}

r = requests.get(url, params=params)
weather = pd.DataFrame(r.json()['daily'])

```

```

weather['time'] = pd.to_datetime(weather['time'])
weather.rename(columns={
    'temperature_2m_max': 'Temp_Max',
    'temperature_2m_min': 'Temp_Min',
    'precipitation_sum': 'Precipitation',
    'windspeed_10m_max': 'WindSpeed'
}, inplace=True)

print("\nWeather data sample:")
print(weather.head())

merged = pd.merge(city_df, weather, left_on='Date Local', right_on='time', how='inner')
merged.drop(columns='time', inplace=True)
print("\nMerged dataset sample:")
print(merged.head())

corr = merged[['NO2 Mean', 'Temp_Max', 'Temp_Min', 'Precipitation', 'WindSpeed']].corr()
plt.figure(figsize=(7, 5))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Correlation Between NO2 and Weather Variables (Houston, 2018–2024)')
plt.show()

print("\nMissing values per column:")
print(merged.isnull().sum())

#Add Lag + Rolling Features

data = merged.copy()
data = data.sort_values('Date Local')
data['NO2_Lag1'] = data['NO2 Mean'].shift(1)

```

```

data['NO2_Lag2'] = data['NO2 Mean'].shift(2)
data['NO2_Lag7'] = data['NO2 Mean'].shift(7)
data['NO2_Rolling7'] = data['NO2 Mean'].rolling(window=7).mean()
data = data.dropna()

```

#Temporal Trend + Seasonality Features

```

data['DayOfYear'] = data['Date Local'].dt.dayofyear
data['Year'] = data['Date Local'].dt.year
data['sin_doy'] = np.sin(2 * np.pi * data['DayOfYear'] / 365)
data['cos_doy'] = np.cos(2 * np.pi * data['DayOfYear'] / 365)

```

```

data['Month'] = data['Date Local'].dt.month

```

```

features = [
    'Temp_Max', 'Temp_Min', 'Precipitation', 'WindSpeed',
    'Month', 'IsWeekend',
    'NO2_Lag1', 'NO2_Lag2', 'NO2_Lag7', 'NO2_Rolling7',
    'DayOfYear', 'sin_doy', 'cos_doy', 'Year'
]
target = 'NO2 Mean'

```

```

train = data[data['Date Local'].dt.year < 2024]
test = data[data['Date Local'].dt.year == 2024]
X_train, y_train = train[features], train[target]
X_test, y_test = test[features], test[target]

```

```

print("\nTrain shape:", X_train.shape, "| Test shape:", X_test.shape)

```

```

def evaluate_model(model, name):

```

```

model.fit(X_train, y_train)
preds = model.predict(X_test)
mae = mean_absolute_error(y_test, preds)
rmse = np.sqrt(mean_squared_error(y_test, preds))
r2 = r2_score(y_test, preds)
print(f"\n{name} Results:\nMAE = {mae:.2f} | RMSE = {rmse:.2f} | R2 = {r2:.3f}")
return preds

```

```

lr = LinearRegression()
rf = RandomForestRegressor(n_estimators=200, random_state=42)
xgb = XGBRegressor(n_estimators=300, learning_rate=0.1, random_state=42)

```

```

pred_lr = evaluate_model(lr, "Linear Regression")
pred_rf = evaluate_model(rf, "Random Forest")
pred_xgb = evaluate_model(xgb, "XGBoost")

```

```

xgb_model = XGBRegressor(random_state=42)
param_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'subsample': [0.7, 0.9, 1.0],
    'colsample_bytree': [0.7, 0.9, 1.0]
}
grid_search = GridSearchCV(
    estimator=xgb_model,
    param_grid=param_grid,
    scoring='r2',
    cv=3,
    verbose=1,

```

```

    n_jobs=-1
)
grid_search.fit(X_train, y_train)
print("Best Parameters:", grid_search.best_params_)
print("Best Cross-Validation R2 Score:", grid_search.best_score_)

best_xgb = grid_search.best_estimator_
y_pred_best = best_xgb.predict(X_test)

mae_best = mean_absolute_error(y_test, y_pred_best)
rmse_best = np.sqrt(mean_squared_error(y_test, y_pred_best))
r2_best = r2_score(y_test, y_pred_best)

print(f"\nTuned XGBoost Results:")
print(f"MAE: {mae_best:.2f}, RMSE: {rmse_best:.2f}, R2: {r2_best:.3f}")

plt.figure(figsize=(10,5))
plt.plot(y_test.values[:100], label='Actual', color='blue')
plt.plot(y_pred_best[:100], label='Predicted (Tuned XGBoost)', color='red', linestyle='--')
plt.title('Actual vs Tuned XGBoost Predicted NO2 Levels (First 100 Days of 2024)')
plt.ylabel('NO2 (ppb)')
plt.legend()
plt.grid(alpha=0.3)
plt.show()

#model stacking

from sklearn.ensemble import StackingRegressor
from sklearn.linear_model import RidgeCV

#base learners

```

```

base_learners = [
    ('lr', LinearRegression()),
    ('rf', RandomForestRegressor(n_estimators=200, random_state=42)),
    ('xgb', XGBRegressor(n_estimators=300, learning_rate=0.05, max_depth=3,
                        subsample=0.7, colsample_bytree=1.0, random_state=42))
]

#defining meta-learner (takes base model predictions as input)
meta_learner = RidgeCV()

#stacked model
stacked_model = StackingRegressor(
    estimators=base_learners,
    final_estimator=meta_learner,
    passthrough=True,
    n_jobs=-1
)

#train and evaluate
stacked_model.fit(X_train, y_train)
stacked_preds = stacked_model.predict(X_test)

mae_stack = mean_absolute_error(y_test, stacked_preds)
rmse_stack = np.sqrt(mean_squared_error(y_test, stacked_preds))
r2_stack = r2_score(y_test, stacked_preds)

print(f"\nSTACKED MODEL RESULTS:")
print(f"MAE: {mae_stack:.2f}, RMSE: {rmse_stack:.2f}, R²: {r2_stack:.3f}")

#comparison plot

```

```

plt.figure(figsize=(10,5))
plt.plot(y_test.values[:100], label='Actual', color='blue')
plt.plot(stacked_preds[:100], label='Stacked Model Prediction', color='green', linestyle='--')
plt.title('Actual vs Stacked Model Predicted NO2 Levels (First 100 Days of 2024)')
plt.ylabel('NO2 (ppb)')
plt.legend()
plt.grid(alpha=0.3)
plt.show()

plt.figure(figsize=(10, 6))
importance = best_xgb.feature_importances_
sorted_idx = np.argsort(importance)
plt.barh(np.array(features)[sorted_idx], importance[sorted_idx], color='skyblue')
plt.title("Feature Importance (Tuned XGBoost)", fontsize=14)
plt.xlabel("Importance")
plt.ylabel("Features")
plt.grid(alpha=0.3)
plt.show()

```